

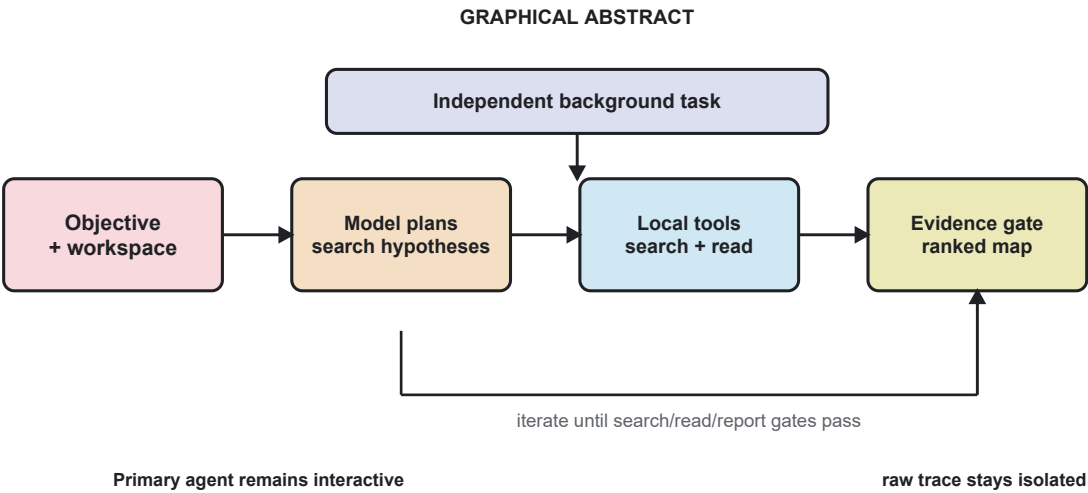
FastContext：面向交互式软件工程智能体的模型驱动异步代码检索架构

FastContext: A Model-Directed Asynchronous Code Retrieval Architecture for Interactive Software Engineering Agents

临界跃迁 TurboFlux 研究团队

技术论文预印本 v1.0，2026 年 7 月 22 日

System snapshot: 5779a946d02106836f60054ec3cd4d27647bddeb



研究定位：本稿是可审计的系统技术预印本。历史基准来自旧架构，只作为先导观察；正式期刊投稿仍需作者信息、目标模板、当前版本多轮实验、统计检验与外部复现。

摘要

大规模代码库中的软件工程任务通常同时包含语义定位、跨文件调用链恢复、证据核验与主上下文预算控制。将整个仓库、宽泛搜索结果或子代理完整工具轨迹直接注入主模型，会造成输入膨胀、缓存失效、低信号干扰以及交互阻塞。本文提出并实现 FastContext，一种面向交互式软件工程智能体的模型驱动异步代码检索架构。该系统将检索规划交给只读语言模型子代理，将精确搜索与文件读取交给本地确定性工具，并通过独立任务生命周期、证据质量门控、紧凑语义代码图和一次性上下文注入实现主 Agent 与检索 Agent 的解耦。FastContext 提供 low、medium 与 max 三档预算；分别约束最大模型轮次、并行工具数、最少搜索与读取次数、推理强度和总运行时限。系统只允许由本轮 read_file 证据支持的文件进入最终 RANKED_CODE_MAP，并要求显式报告执行流、已尝试搜索和不确定性。异步调度层使用独立 AbortController、运行时任务状态机与追加式 JSONL 转录，主会话中断不会误杀后台 FastContext；退出、显式取消与硬超时仍可回收任务。本文给出完整架构、业务分支、实现映射和设计依据，并审慎复核一组旧版本单轮先导基准。该先导实验在 8 个仓库内任务上报告了 FastContext 94.8 与 Claude Code Explore 69.9 的端到端自定义质量指数，但实验来自包含已删除自动预扫描的历史提交，样本量为每任务一次，因此不能代表当前实现，也不足以支持统计显著或普遍优越性结论。本文的主要贡献是一个可审计、可取消、上下文隔离且面向工程证据的代码检索系统设计，以及一套避免将实现演进误写为实验结论的报告方法。

关键词：软件工程智能体；代码检索；工具增强语言模型；异步子代理；上下文隔离；证据门控

Abstract

Repository-scale software engineering requires semantic localization, cross-file execution-flow reconstruction, evidence verification, and strict control of the primary agent's context budget. Injecting a repository snapshot, broad search dump, or complete subagent trace into the primary context increases input cost, disrupts prompt caching, and introduces low-signal evidence. We present FastContext, a model-directed asynchronous retrieval architecture for interactive software engineering agents. A read-only language-model subagent owns query planning and evidence selection, while deterministic local tools execute only requested searches and bounded file reads. The design combines an independently cancellable runtime task, evidence-grounded quality gates, a compact ranked code map, and one-shot context injection. Three operating levels jointly control turns, tool parallelism, required searches and reads, native reasoning effort, and wall-clock deadlines. Only files read by the subagent in the current run may appear in the authoritative report, which must include execution flow, searches tried, and residual uncertainty. A historical single-round pilot benchmark is reported solely as non-current evidence because it predates the removal of eager deterministic prefetch. The paper contributes an auditable architecture and a reproducible reporting protocol rather than a claim of statistically established superiority.

Keywords: software engineering agents; code retrieval; tool-augmented language models; asynchronous subagents; context isolation; evidence grounding

1 引言

软件工程 Agent 的困难并不只在生成代码。真实任务往往从一句含糊的自然语言开始，例如“输入框为什么卡死”“审批状态在哪里更新”或“中断后的流式文本为何消失”。要回答这些问题，系统必须先找到真正的入口、实现核心、调用者、状态边界、失败路径与测试，再决定是否编辑。SWE-bench 表明，真实 GitHub 问题通常跨越多个函数、类和文件，并需要与执行环境交互 [6]。RepoCoder 进一步说明，仓库级代码任务需要迭代式检索与生成，而不是仅依赖当前文件 [5]。

最直接的方案是预先扫描仓库并把候选片段全部送入模型。这种方案在小型仓库中可能提高召回，但在宽泛工作区中会产生两个问题。第一，多个无目标搜索可占用 CPU、文件系统与子进程资源；第二，被选中的低置信片段仍会进入模型输入，形成固定 Token 税。Self-RAG 对“无差别检索固定数量段落”的批评同样适用于代码场景：检索是否发生、检索什么以及何时停止，应由任务需要决定，而不是由固定预取流程决定 [4]。

FastContext 的核心判断是：本地工具擅长快速、精确、可复现地执行搜索与读取；语言模型擅长把模糊目标改写为搜索假设、判断证据角色、恢复调用关系并发现反例。因此系统不在模型前运行自动预扫描，而让模型按

需调用 search_content、search_files、search_symbols、get_codemap 与 read_file。该设计与 ReAct 的“推理-行动-观察”循环 [2]、Toolformer 的工具选择思想 [3] 以及 Claude Code 将 Explore 放入独立上下文的工程实践 [11] 一致，但额外加入了严格读取门控、分档预算、后台生命周期隔离和一次性证据注入。

本文研究对象为 TurboFlux CLI 主分支提交 5779a946d02106836f60054ec3cd4d27647bddeb。贡献如下：

- 提出模型驱动、只读、异步的代码检索子代理，将语义决策与本地确定性执行分离。
- 设计强制搜索、强制读取、报告结构校验和反例搜索组成的证据门控，降低“只看文件名就下结论”的风险。
- 设计独立 AbortController、任务状态机、硬超时、转录持久化和主会话中断隔离，保证后台检索不阻塞主交互。
- 设计紧凑 RANKED_CODE_MAP 与一次性注入协议，阻止原始工具轨迹污染主上下文。
- 给出源码级实现映射、形式化评分、三档预算与历史先导实验的效度边界。

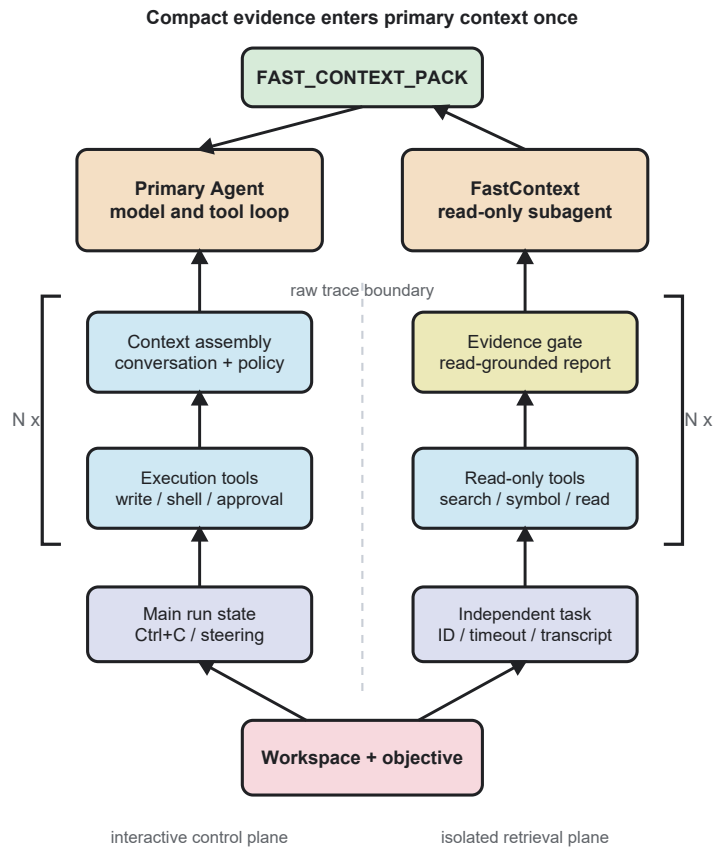


图 1. FastContext 双上下文架构。模型驱动检索平面与主交互平面共享工作区工具，但原始轨迹隔离，仅紧凑证据包单向进入主上下文。

2 问题定义与设计目标

给定工作区 W 、自然语言目标 q 和预算级别 l ，代码检索系统需要输出一个按相关性排序的证据集合 M 。每个候选项包含路径 p 、行区间 $[a,b]$ 、角色 r 、置信度 c 与理由 e 。系统不直接生成修改，而为主 Agent 提供可再次核验的代码图。

形式化地，FastContext 求解：

$$M = F(q, W, l) = \text{Rank}(\text{Ground}(\text{Trace}(\text{Plan}(q), \text{Tools}(W))), l)$$

其中 Plan 由语言模型生成独立搜索假设；Tools 只执行被请求的本地操作；Trace 恢复 entry/caller 到 implementation 再到 state、persistence、test 或 failure path 的关系；Ground 要求最终候选至少被本轮 read_file 直接读取；Rank 生成 3 至 7 个主候选并显式保留不确定性。

系统设计目标为：G1 高信号定位；G2 主上下文隔离；G3 主 Agent 持续可交互；G4 可取消且有上界；G5 供应商与模型协议兼容；G6 结果可追溯；G7 在小任务上不过度代理。非目标包括通用向量数据库、长

期语义索引、自动代码修改以及对任何模型或产品的未经重复实验的优越性声明。

3 相关工作与工程参考

3.1 检索增强与工具增强语言模型

RAG 将参数化模型与非参数化检索结合，以提高知识密集任务的事实性和来源可追溯性 [1]。ReAct 将推理轨迹与环境行动交错，使模型能够根据观察修正后续步骤 [2]。Toolformer 研究模型何时调用工具、传递什么参数以及如何吸收返回值 [3]。FastContext 采用相同的基本分工，但检索对象是活动代码仓库，工具输出包含路径、行号和源码片段，最终产物不是自然语言答案，而是供另一个 Agent 消费的受约束代码图。

Self-RAG 指出固定、无差别检索可能降低质量 [4]。这直接支持 FastContext 取消自动预扫描：语言模型先根据目标选择查询，再由本地工具精确执行。RepoCoder 的迭代检索-生成循环 [5] 说明一次性静态候选通常不足以覆盖仓库级依赖。FastContext 的每轮模型请求都可根据上轮搜索与读取结果调整下一轮查询。

3.2 软件工程 Agent 与代码定位

SWE-bench [6] 将仓库级问题定位、编辑与测试置于统一评估环境。Agentless [7] 则证明简化的定位-修复-验证流水线可以与复杂 Agent 竞争，提醒系统设计者不要为自主性而自主。FastContext 因此只负责定位与理解，不写文件、不运行破坏性工具，并通过 low 档支持快速定向定位。精确字符串或已知符号应直接使用本地搜索，不必启动子代理。

3.3 工业实现参考

Claude Code 官方文档把 Explore 定义为独立上下文、只读、继承主模型并支持 quick、medium、very thorough 三种深度的内置子代理 [11]。其价值主张是把搜索结果、日志和文件内容留在子上下文，只返回摘要。TurboFlux 借鉴了这种隔离与分档思想，但将档位命名为 low、medium、max，并增加最少搜索/读取约束和结构化报告校验。

OpenCode 在 TaskTool 中为子代理建立 child session，并由 BackgroundJob 托管异步执行、取消和完成结果注入 [12]。Claude Code 本地源码快照还显示异步 Agent 使用与父线程解除链接的 AbortController。FastContext 据此将后台控制器从主 Agent 中断链路中分离；主 Ctrl+C 只中断主 run，显式 cancel_agent、CLI 销毁和任务硬超时仍可终止 FastContext。

需要强调的是，FastContext 不是 Claude Code 或 OpenCode 的复制。前者没有在公开文档中承诺本文的读取次数门控和 RANKED_CODE_MAP 协议；后者的通用 child session 也不等同于 FastContext 的代码证据评分。参考实现用于验证生命周期边界，而核心检索协议、事件模型与一次性注入由 TurboFlux 实现。

来源	采用的思想	FastContext 的差异
ReAct / Toolformer	模型决定工具与参数，并根据观察迭代	工具域限制为只读代码证据；增加读取与报告门控
Self-RAG / RepoCoder	按需、迭代检索，而非固定一次性上下文	无自动预扫描；最终产物是代码图而非答案或补全
Claude Code Explore	独立上下文、只读、继承模型、三档深度	low/medium/max；显式最少搜索/读取与结构校验
OpenCode TaskTool	child session、后台 Job、取消与完成通知	进程内 RuntimeTask + JSONL transcript + 一次性 pack

4 系统架构

4.1 双上下文与单向证据通道

图 1 展示主 Agent 与 FastContext 的双通道结构。主 Agent 保留用户对话、任务规划、写工具和审批状态；FastContext 拥有独立的消息数组、系统提示、工具调用历史和证据账本。两者共享只读 ToolExecutor 与工作区边界，但不共享原始搜索轨迹。FastContext 完成后仅生成 fast_context_pack，其中权威部分最多保留约 5,000 字符的 LLM 语义报告。该 pack 在下次主模型上下文构造时注入，并立即从运行时缓存清除，避免每个后续 turn 重复计费。

这一设计与把整个子代理 transcript 复制进主会话不同。transcript 仍以 JSONL 保存在 .turboflux/runtime-agents 中，可由 read_agent 分页读取，用于审计和故障恢复；默认主上下文只得到代码图。由于中文与代码 Token 化比例依赖具体模型，系统对字符数而非 Token 数设置硬上限，论文不把 5,000 字符换算成固定 Token 数。

4.2 异步调度与去重

startFastContextBackground 首先规范化 objective。空目标或无工作区返回 unavailable；若已有任务，目标相同返回 running，目标不同返回 busy；否则创建独立 AbortController、递增 generation、注册 RuntimeTask 并立即返回 started 与 taskId。单实例 fastContextRunPromise 形成并发去重屏障，防止模型重复调用 explore_code 产生重叠扫描。

generation 用于抑制过期任务事件和过期证据注入。任务完成后仅在 promise 身份仍匹配时清理运行槽。主 Agent 的 abort 不再触碰后台 FastContext 控制器；standalone FastContext 仍受当前命令中断控制。Engine destroy 会回收后台控制器，RuntimeTaskManager 则负责 completed、failed、stopped 与 interrupted 终态。

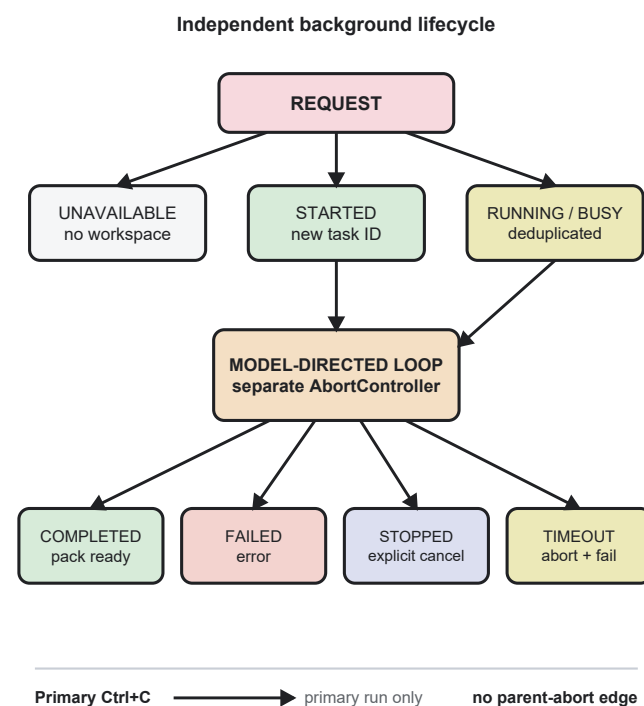


图 3. 后台生命周期。主会话 Ctrl+C 与 FastContext 控制器之间不存在父中断边；显式取消、销毁和硬超时仍可终止任务。

4.3 模型驱动检索循环

FastContext 的首个动作是模型请求，而不是脚本预取。子代理收到 objective、深度合同与五类只读工具。每轮模型可以并行发出超过 maxParallel 的工具调用。工具结果进入子代理消息历史和证据账本，随后模型决定继续搜索、读取还是综合。该循环类似 ReAct，但行动空间被限制为代码检索与读取。

search_content 使用 ripgrep 执行分页正则检索；search_files 使用文件 glob；search_symbols 将多语言声明模式合并为一次 ripgrep 扫描；get_codemap 提供层级方向；read_file 执行有界行范围读取。AppData、node_modules、构建产物和常见缓存目录被排除。工具均在 workspace sandbox 内解析路径。

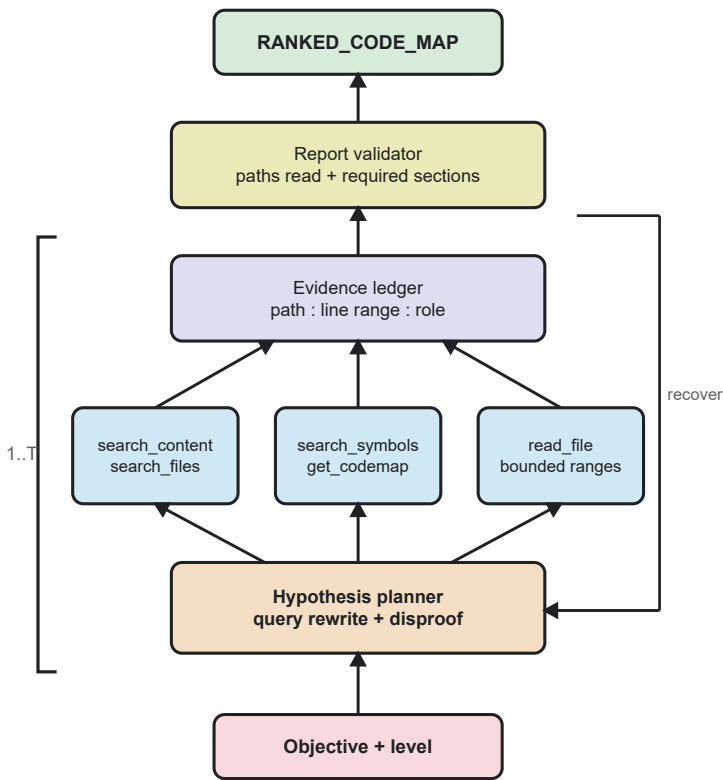


图 2. 模型驱动检索与证据门控。循环上界 T 由档位决定；不足的搜索、读取或报告结构触发受限恢复。

4.4 证据质量门控

FastContext 不接受“文件名看起来像”作为最终证据。每档配置 minimumSearchCalls 与 minimumReadCalls。若模型在达到最少搜索前尝试结束，运行器追加恢复提示并要求替代标识符、引用、文件名或运行时术语；若已有候选但读取不足，则要求对最强候选执行 read_file。若首轮没有证据，则只允许一次改写查询的 recovery search。

最终文本必须以 RANKED_CODE_MAP 开始，并包含 EXECUTION_FLOW、SEARCHES_TRIED 与 UNCERTAINTY。至少一个本轮 read_file 路径必须出现在报告中。第一次结构校验失败会触发一次 report recovery；再次失败则返回错误而不是伪造合格报告。此设计把“模型自信”替换为可机器检查的最低证据契约。

4.5 后验本地评分

模型最终报告是主排序权威。仅当报告缺失或不可用时，FastContext 才对已经由模型工具调用产生的证据做后验评分；该评分不会发起搜索，也不会把额外文件加入证据集。对证据 h ，基础分为：

$$s(h) = \text{clip}(w_{\text{role}} + 8m_{\text{path}} + 4m_{\text{text}} + w_{\text{source}} + a_{\text{entry}} - p_{\text{span}} - p_{\text{doc}}, 20, 140)$$

角色基础权重从 root_cause 的 78 到 supporting 的 44；文件读取、符号命中与 codemap 分别获得不同来源加分；超过 90 行的宽片段和与目标无关的文档受到惩罚。文件级分数再叠加证据角色多样性、密度与读取确认奖励，并对仅包含测试的候选降权。该规则用于 degraded 输出的保守排序，不替代 LLM 的语义执行流。

5 业务逻辑分支

FastContext 有两类入口。explore_code 是主 Agent 的语义检索工具，用于未知功能区、跨文件行为、命名不确定或一次定向搜索不足的任务；spawn_agent(agent_type=fast_context) 是统一子代理入口。精确符号、确定字符串或已知路径应直接使用 search_symbols、search_content 或 read_file，以避免代理开销。

条件	返回/动作	主 Agent	终止机制
空目标或无工作区	unavailable	继续定向工具或提示工作区	无任务
同目标已运行	running + taskId	不重复派遣	原任务控制器
不同目标已运行	busy + active objective	避免重叠检索	原任务控制器
新目标	started + taskId	立即继续，不 await	cancel/destroy/timer out
报告合格	completed + pack	下一模型轮次一次性注入	自动清槽
模型/门控失败	failed	显示 warning，使用定向工具	自动清槽

主 Agent 启动 FastContext 后不等待 promise。工具结果明确指示主模型继续执行定向搜索/读取，FastContext 完成证据将在后续模型轮次一次性注入。用户可以继续输入 steering message；Ctrl+C 只中断主 run。用户需要停止后台任务时使用 cancel_agent。若工作区关闭或 CLI 销毁，engine destroy 回收控制器。任务超过 low 180 秒、medium 360 秒或 max 720 秒时，SubAgentTaskManager 先拒绝有界 promise，再 abort 控制器并把任务记为 failed。

FastContext 跟随当前主模型与 API 配置，而不是维护第二套隐式模型。请求层根据 provider 和模型规划 Anthropic Messages、OpenAI Responses 与 OpenAI Chat Completions 的协议候选，支持瞬态网络重试与不兼容参数降级。单次模型请求默认上限为 90 秒，任务级时限提供更外层的终止保证。

6 三档预算设计

档位	轮次	并行	搜索/读取	推理	总时限
low	5	4	1/2	low	180 s
medium	8	6	2/3	medium	360 s
max	12	8	4/6	max	720 s

low 适合定位入口、可见文案或单一实现，要求至少两个假设，读取 2 至 4 个候选后尽快停止。medium 面向常规工程任务，要求至少三个假设并恢复 caller-to-core 与状态/配置边界。max 面向架构、复杂故障和完整链路，要求至少四个独立假设、测试与失败路径，以及主动寻找反证。

档位同时控制计算深度和工程风险，而不是只控制“思考更久”。max 提高并行工具数可能增加瞬时 I/O，因此总时限与 UI 事件上限同时存在。UI 每 80 ms 批量刷新 FastContext 事件，只保留最近 120 条事件；累计文件、命中和阶段摘要单独归约，避免长检索导致 Ink 高频重绘和内存无界增长。

7 实现与可追溯性

模块	职责	关键技术
agentEngine.ts	入口、去重、generation、pack 注入	Promise 槽；独立 AbortController
fastContextSubagent.ts	事件映射、证据角色、紧凑代码图	后验评分；5,000 字符报告上限
subAgent.ts	模型循环、协议适配、工具执行、质量门控	ReAct 式循环；并行 Promise.all
SubAgentTaskManager	后台任务、超时、JSONL transcript	Promise.race；append-only journal
RuntimeTaskManager	统一运行状态与 stop control	状态机；事件发布
NodeToolExecutor	搜索、符号、读取、sandbox	ripgrep；路径约束；目录排除
FastContext UI	阶段、worker、证据和摘要	80 ms 批处理；120 事件环形窗口

SubAgentTaskManager 为每个任务生成稳定 ID、记录 objective、workspace、ownerSessionId 与 transcriptPath。转录采用追加式 JSONL，记录 start、event、result 和 state。重启时，已完成任务恢复结果；没有终态的旧任务被标记 interrupted，而不是错误显示为仍在运行。RuntimeTaskManager 提供统一状态与 stop control，使 FastContext、普通 Agent 与终端任务共享生命周期语义。

FastContext 事件类型包括 phase、worker、file、hit 和 insight。CLI 把事件映射为 MAPPING、RANKING、SYNTHESIZING、DONE 或

ERROR, 展示 wave、文件数、证据区间与活动 worker。事件流既是用户反馈, 也是 transcript 审计数据, 但不进入主模型上下文。

安全方面, FastContext 定义为只读 Agent, 工具注册表把 explore_code 与 spawn_agent(fast_context) 标为 read-only、non-destructive。路由由 ToolExecutor 约束在 workspace; 环境密钥不属于检索目标; 原始 transcript 以 0600 模式写入支持该权限的系统。本文不声称该层可替代操作系统级沙箱或企业数据治理。

8 设计选择与替代方案

8.1 为什么取消自动预扫描

旧实现会在模型前并行执行多组文件、内容和符号搜索, 再把候选片段注入子代理。这提高了固定查询的冷启动召回, 但在 C:\Users\Administrator 这类宽工作区中会扫描 AppData 与多个项目, 造成长时无模型进展。更重要的是, 预取片段无论是否真正相关都进入输入, 形成固定成本。当前实现删除该阶段, 只保留模型主动调用的本地工具。这个选择与 Self-RAG 的按需检索原则一致 [4], 也更接近 Claude Code Explore 的公开描述 [11]。

8.2 为什么不使用纯本地搜索

纯 ripgrep 对精确标识符非常有效, 但用户目标常以业务语言、UI 现象或跨模块行为表达。脚本难以稳定判断“入口、状态持久化、错误路径和测试”之间的关系。FastContext 让模型负责查询改写和执行流恢复, 让本地工具负责精确执行。对于已知符号或字符串, 系统仍明确要求绕过子代理, 直接调用本地工具。

8.3 为什么不把完整 transcript 返回主 Agent

完整轨迹包含重复搜索、失败查询、工具参数和大段源码。直接注入会让主模型重新解释低层过程, 并破坏后续缓存。紧凑 pack 保留目标、轮次、耗时、读取证据数量、最终代码图与不确定性; 原始 transcript 可按需审计。这对应 Claude Code 所述“子代理独立上下文只返回摘要”的上下文管理动机 [11]。

8.4 为什么后台任务不继承主中断

用户常在主 Agent 输出过长或方向不对时按 Ctrl+C, 但仍希望已派遣的检索继续。若共享父 AbortController, 主中断会把后台任务一起清除, 造成“子代理存在但不独立”的假象。Claude Code 源码快照明确区分同步共享与异步 unlinked controller; OpenCode 使用 BackgroundJob 和 child session [12]。FastContext 因此把主中断与后台取消分开, 但保留显式 cancel、destroy 和 timeout 三条回收路径。

9 先导实验

9.1 数据来源与实验设置

仓库保存了一组 2026-07-21 生成的受控对比数据。其源提交为 629e4c25bc646c98113cddca4c86622a286cffdc, TurboFlux 0.1.5 medium 对比 Claude Code 2.1.177 Explore, 二者通过同一 Anthropic 兼容端点使用 claude-sonnet-5, 原生 reasoning 均关闭。8 个任务按 AB/BA 交替顺序各运行一次, 单例超时 240 秒。人工整理参考文件, 计算 Recall@5、Recall@10、MRR、Top-1、行引用率和执行流章节完成率。自定义质量指数为:

$$Q = 60 R@10 + 25 MRR + 10 C + 5 E$$

其中失败或超时记 0。该指数是透明的工程复合指标, 不是同行评审标准, 也未验证各权重的外部效度。

9.2 历史结果

指标	旧 FastContext	Claude Code	解释
成功率	100%	75%	Claude 两次 240 s 超时
Recall@10	0.927	0.677	失败运行计 0
MRR	1.000	0.750	失败运行计 0
端到端 Q	94.8	69.9	自定义复合指数
成功案例 Q	94.8	93.2	质量接近
成功延迟 p50	66.8 s	107.0 s	单轮观察
成功延迟 p95	107.0 s	208.7 s	非统计估计

历史 FastContext 完成 8/8, Claude Code 完成 6/8; 端到端 Q 为 94.8 对 69.9。仅比较成功案例时, Q 为 94.8 对 93.2, 差异显著缩小, 说明端到端差距主要来自两次超时, 而不是成功结果质量的普遍优势。历史 FastContext 成功延迟 p50 为 66.8 秒、p95 为 107.0 秒; Claude Code 为 107.0 秒与 208.7 秒。成功案例平均输入/输出 Token 分别为 1310/1645 与 949/2429, 但 Claude Code 超时运行缺少最终 usage, 因此这些数值不能解释为总成本。

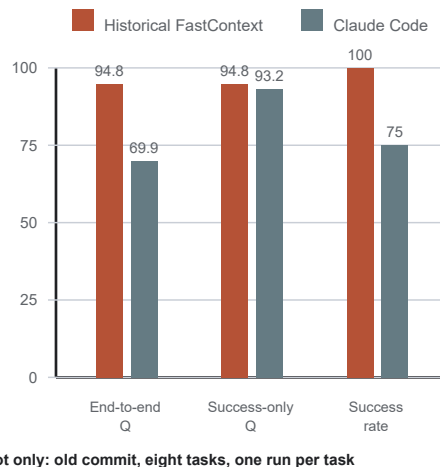


图 4. 历史先导实验。数据来自已删除自动预扫描的旧提交, 仅用于描述历史观察, 不代表当前系统。

9.3 为什么这些结果不能代表当前版本

历史提交的 executionModel 明确为“deterministic prefetch plus LLM subagent retrieval”。当前提交已经删除自动预扫描、改变首轮输入、错误恢复提示和宽工作区行为。因此历史结果只能证明旧原型在该机器、该中转 API、该仓库和该次采样中的表现, 不能用于声称当前 FastContext 超过 Claude Code。要评价当前版本, 必须重新运行多轮、跨仓库、随机化顺序并报告置信区间。

当前版本的确定性验证仅包括代码级测试: FastContext 不在模型前调用本地搜索; 父 Agent abort 不终止后台 FastContext; 任务超时会 abort 并释放运行槽; 事件缓冲有界; 分档预算单调增加。全仓库测试在 2026-07-22 通过 57 个测试文件、492 个测试。该结果证明实现一致性, 不证明检索质量。

10 威胁效度与未解决问题

内部效度方面, 历史实验每任务只有一次运行, 无法估计模型随机性、网络抖动或中转站排队效应。两系统虽使用同一模型标识, 但请求协议、系统提示、缓存命中和工具实现不同。人工参考文件可能遗漏合法支持文件, 复合质量指数权重也由项目定义。

外部效度方面, 8 个任务全部来自 TurboFlux 自身仓库, 系统对自身命名和模块结构可能具有优势。尚未覆盖大型多语言单体仓库、生成代码、子模块、稀疏检出、非 Git 工作区和企业权限边界。中文 UI 文案任务占比也可能放大特定提示策略的收益。

构造效度方面, Recall 与 MRR 衡量文件定位, 不等同于修正准确率。执行流章节存在不表示链路内容完全正确; read_file 证据也可能被误解。未来应引入调用边校验、符号级命中、补丁成功率、测试通过率和人工盲评。

当前架构仍有四个工程问题。第一, 后台结果采用下一模型轮次注入; 若用户已切换主题, 可能出现证据陈旧, 需要 objective 相似度或显式接受机制。第二, FastContext 与主模型共享 API 端点, 受限

中转站可能发生并发竞争，需要优先级队列和连接配额。第三，进程退出后任务只恢复 transcript，不跨进程继续执行，完整持久化需要 Runtime Daemon。第四，5,000 字符上限按字符而非 tokenizer 预算，未来应按当前模型动态裁剪。

11 复现实验路线

正式投稿前应执行以下协议：至少选择 5 个语言生态、20 个公开仓库和 100 个定位任务；每个系统每任务运行不少于 5 次；对顺序、缓存冷热和模型端点分层随机化；预注册参考文件与评分脚本；同时报告成功率、Recall@K、MRR、符号命中、调用边准确率、延迟、实际计费 Token 和成本；对比例指标使用 bootstrap 置信区间，对配对结果使用适当的非参数检验；公开所有失败 transcript，并进行三组消融：无读取门控、共享父中断、无一次性压缩。只有完成该协议，才可讨论“超过 Claude Code”或“工业级领先”。

12 结论

FastContext 将代码检索从“先扫仓库再喂模型”改造成“模型规划、工具执行、证据核验、紧凑回传”的异步子代理系统。其核心价值不在某个搜索算法，而在边界：模型只决定语义，本地只执行精确操作；后台任务不占用主交互控制权；没有读取证据就不能形成权威代码图；原始工具噪声不进入主上下文；失败、取消与超时都有可观测终态。当前实现已经形成清晰、可测试的系统架构，但检索优势仍需新版本、多仓库、多轮实验验证。本文因此将历史性能数据定位为先导证据，把可审计设计与诚实效度边界作为主要贡献。

致谢与披露

FastContext 属于 TurboFlux CLI。论文由项目源码、测试、历史基准数据、Claude Code 官方文档、OpenCode 官方源码与公开学术文献整理。本文未运行新的付费模型实验，未编造性能数据。Claude Code 本地源码快照仅用于实现对照；可公开复核的产品事实优先引用官方文档。作者与机构信息、利益冲突、伦理声明和数据可用性声明应在正式投稿前按目标期刊模板补全。

参考文献

[1] Lewis, P., Perez, E., Piktus, A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020. arXiv:2005.11401.

[2] Yao, S., Zhao, J., Yu, D., et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. arXiv:2210.03629.

[3] Schick, T., Dwivedi-Yu, J., Dessi, R., et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023. arXiv:2302.04761.

[4] Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR, 2024. arXiv:2310.11511.

[5] Zhang, F., Chen, B., Zhang, Y., et al. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. EMNLP, 2023. arXiv:2303.12570.

[6] Jimenez, C. E., Yang, J., Wettig, A., et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. arXiv:2310.06770.

[7] Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024.

[8] Robertson, S., and Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4):333-389, 2009.

[9] Manning, C. D., Raghavan, P., and Schütze, H. Introduction to Information Retrieval. Cambridge University Press, 2008.

[10] Vaswani, A., Shazeer, N., Parmar, N., et al. Attention Is All You Need. NeurIPS, 2017. arXiv:1706.03762.

[11] Anthropic. Create custom subagents - Claude Code Docs. <https://code.claude.com/docs/en/sub-agents>, accessed 2026-07-22.

[12] Anomaly Co. OpenCode TaskTool and BackgroundJob implementation, commit 0a601cf334b9a83cc2854108a2b860f25c6e7e8e. <https://github.com/anomalyco/opencode>, accessed 2026-07-22.

[13] BurntSushi. ripgrep: recursively search directories for a regex pattern. <https://github.com/BurntSushi/ripgrep>, accessed 2026-07-22.

[14] OpenJS Foundation. Node.js AbortController and AbortSignal API. <https://nodejs.org/api/globals.html#class-abortcontroller>, accessed 2026-07-22.

[15] TurboFlux Research Team. TurboFlux CLI source snapshot, commit 5779a946d02106836f60054ec3cd4d27647bddeb, 2026.

附录 A 可复现性清单

- 研究对象提交：5779a946d02106836f60054ec3cd4d27647bddeb。
- 历史实验提交：629e4c25bc646c98113cddca4c86622a286cffdc。
- 当前核心模块行数：agentEngine.ts 6025；fastContextSubagent.ts 488；subAgent.ts 1190；SubAgentTaskManager 394；RuntimeTaskManager 249；FastContextBanner 214；fastContextUi 60。
- 当前分档：low 5 turns/4 parallel/1 search/2 reads/180 s；medium 8/6/2/3/360 s；max 12/8/4/6/720 s。
- 单请求超时：FastContext 90 s；协议候选：Anthropic Messages、OpenAI Responses、OpenAI Chat Completions。
- 最终语义报告字符上限：5,000；fallback 候选上限：12；UI 最近事件上限：120；UI 刷新批次：80 ms。
- 测试命令：npm test；类型检查：npm run type-check；构建：npm run build。
- 历史数据：benchmark-results/2026-07-21-claude-sonnet-5/comparison-d ata.json。

附录 B 算法伪代码

Algorithm 1 StartFastContext(q, level)
1: q ← trim(q) 2: if q is empty or workspace is absent: return UNAVAILABLE 3: if activePromise exists and activeObjective = q: return RUNNING 4: if activePromise exists: return BUSY(activeObjective) 5: controller ← new AbortController() // not linked to primary run 6: generation ← generation + 1 7: task ← Runtime.start(kind=fast_context, timeout=budget[level]) 8: activePromise ← task.run(ModelDirectedRetrieve(q, level)) 9: on settle: clear slot iff activePromise identity still matches 10: return STARTED(task.id)
Algorithm 2 ModelDirectedRetrieve(q, level)
1: messages ← [system(depth contract), user(q)] 2: evidence ← empty ledger; searches ← 0; reads ← 0 3: for turn = 1..maxTurns[level]: 4: response ← model(messages, read-only tools) 5: if response has tool calls: 6: execute at most maxParallel[level] calls concurrently 7: append observations; normalize and deduplicate evidence 8: continue 9: if searches or reads below minimum: append recovery instruction 10: else if report schema or read grounding fails: recover once 11: else return compact RANKED_CODE_MAP 12: return FAILED or TRUNCATED with explicit uncertainty